

Announce Finder

Name: Simu Darius

Group: 30435

Contents

1	Deliverable 1	3
1.1	Project Specification	3
1.2	Functional Requirements	3
1.3	Use Case Model	3
1.4	UML Use Case Diagrams	5
1.5	Supplementary Specification	5
1.5.1	Non-functional Requirements	5
1.5.2	Design Constraints	6
1.6	Glossary	6
2	Deliverable 2	7
2.1	Domain Model	7
2.2	Conceptual Architecture	8
2.3	Package Design	9
2.4	Components and Deployment	9
3	Deliverable 3	10
3.1	Design Model	10
3.1.1	Dynamic Behavior	10
3.1.2	Class Diagram	11
3.1.3	Data Model	11
3.1.4	System Testing	12
3.1.5	Future Improvements	12
3.2	Conclusion	13
3.3	Bibliography	14

1 Deliverable 1

1.1 Project Specification

AnnounceFinder is a web-based application that aggregates product listings and announcements from multiple online marketplaces into a single unified search interface. The system crawls and scrapes data from both second-hand platforms as well as brand new retail websites, allowing users to search for any type of product or item. Results are presented in a consistent format including title, price, platform, image, and a direct link to the original listing. The user may choose how products are ordered: depending on price, platform, or type: new or used. Registered users can authenticate and save listings to a personal favorites list stored in a database.

1.2 Functional Requirements

- FR1: The system shall allow users to search for any product or item by entering a keyword in a search field.
- FR2: The system shall query multiple data sources simultaneously and aggregate the results.
- FR3: The system shall display search results including title, price, platform, image, and a link to the original listing.
- FR4: The system shall filter results to ensure relevance to the search query using keyword and fuzzy matching.
- FR5: The system shall allow users to choose how the items are ordered: based on price, platform or type: new or used.
- FR6: The system shall allow users to register and log in with a username and password.
- FR7: Authenticated users shall be able to save listings to a personal favorites list.
- FR8: Authenticated users shall be able to view and manage their saved favorites.
- FR9: The system shall respect robots.txt rules for all scraped websites.
- FR10: The system shall enforce rate limiting between requests to avoid overloading external servers.

1.3 Use Case Model

Use-Case: Search for listings

Level: User goal

Primary Actor: Guest or registered user

Main success scenario:

1. User enters a search query in the search field.
2. System sends the query to all registered scrapers.
3. Each scraper fetches results from its respective platform.
4. System filters results using keyword and fuzzy matching.
5. System aggregates and returns the results.
6. Results are displayed to the user with title, price, platform, and image.

Extensions:

1. A scraper fails or is blocked by robots.txt:
 - (a) The scraper returns an empty list and logs the error.
 - (b) Results from remaining scrapers are still displayed.
2. Query is empty:
 - (a) System returns no results and prompts the user to enter a keyword.

Use-Case: Register and log in

Level: User goal

Primary Actor: Guest user

Main success scenario:

1. User navigates to the registration page.
2. User enters a username and password.
3. System validates the input and creates an account.
4. User logs in with their credentials.
5. System authenticates the user and starts a session.

Extensions:

1. Username already exists:
 - (a) System notifies the user and prompts for a different username.
2. Password does not meet requirements:
 - (a) System displays password requirements and asks the user to retry.

Use-Case: Save a listing to favorites

Level: User goal

Primary Actor: Registered user

Main success scenario:

1. User performs a search and receives results.
2. User clicks the “Save to Favorites” button on a listing.
3. System stores the listing details in the database linked to the user.
4. System confirms the listing was saved.

Extensions:

1. User is not logged in:
 - (a) System redirects the user to the login page.
2. Listing already saved:
 - (a) System notifies the user the listing is already in their favorites.

1.4 UML Use Case Diagrams

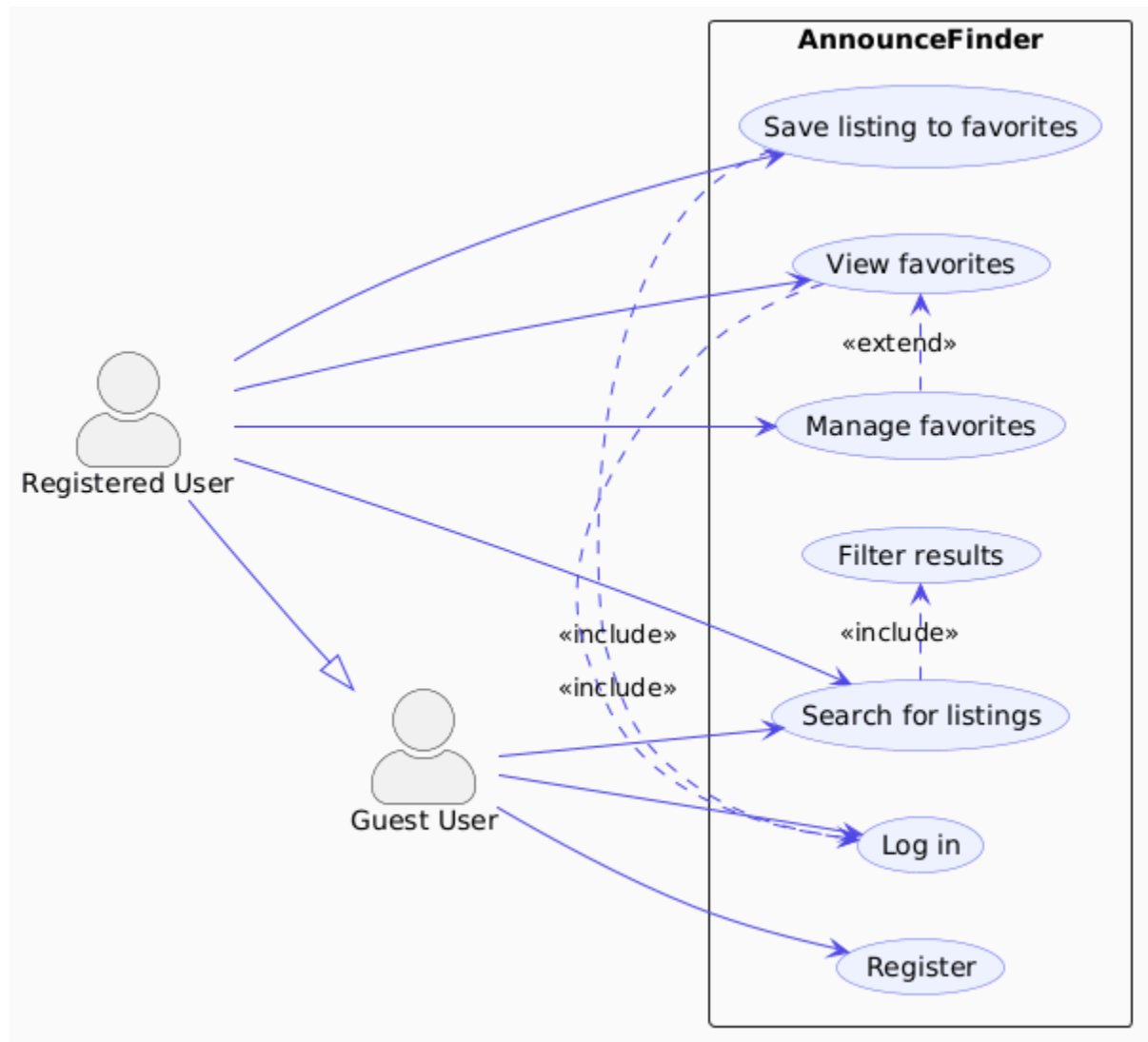


Figure 1: Use Case Diagram

1.5 Supplementary Specification

1.5.1 Non-functional Requirements

- **NFR1 - Performance:** The system shall return search results within 10 seconds for a standard query across all platforms. This is necessary because scraping involves multiple sequential network requests and users should not wait excessively for aggregated results.
- **NFR2 - Reliability:** The system shall continue to function and return partial results even if one or more scrapers fail. This is achieved through the plugin architecture where each scraper is isolated and exceptions do not propagate to the rest of the system.
- **NFR3 - Security:** User passwords shall be stored hashed in the database and never in plain text. Sessions shall be managed securely to prevent unauthorized access to

user favorites.

- NFR4 - Maintainability: The system shall be structured so that new scrapers for additional platforms can be added or existing ones removed without modifying the core application logic. This is achieved through the BaseScraper plugin architecture where each platform is encapsulated in its own class.

1.5.2 Design Constraints

- DC1: The system is built using Python and the Flask web framework, which constrains the backend technology stack.
- DC2: Scraping is limited to publicly accessible pages only. Pages requiring authentication or blocked by robots.txt are excluded.
- DC3: The eBay integration relies on the official eBay Browse API, which requires a registered developer account and is subject to eBay's API usage policies and rate limits.
- DC4: The system runs as a local development server and is not deployed to a production environment in this iteration.
- DC5: Rate limiting is enforced per domain to comply with website terms of service, which limits the speed at which results can be retrieved.
- DC6: Scrapers that rely on HTML parsing are subject to breaking if the target website changes its HTML structure.

1.6 Glossary

- Web Crawler: A software program that automatically browses the web, following links and collecting data from web pages.
- Web Scraper: A program that extracts specific data from the HTML content of a web page.
- Listing: A single product announcement retrieved from a marketplace, containing title, price, platform, image, and URL.
- BaseScraper: The abstract base class that all platform-specific scrapers inherit from, defining the common interface for the search method.
- robots.txt: A file hosted by websites that specifies which pages crawlers are allowed or disallowed from accessing.
- Rate Limiting: The practice of introducing delays between requests to avoid overloading external servers.
- Fuzzy Matching: A technique for comparing strings that allows for partial or approximate matches, used to improve the relevance of search results.
- API (Application Programming Interface): A structured interface that allows software to request data from an external service in a structured format, as opposed to scraping raw HTML.
- Favorites: A user-specific collection of saved listings stored in the database, accessible only after authentication.

2 Deliverable 2

2.1 Domain Model

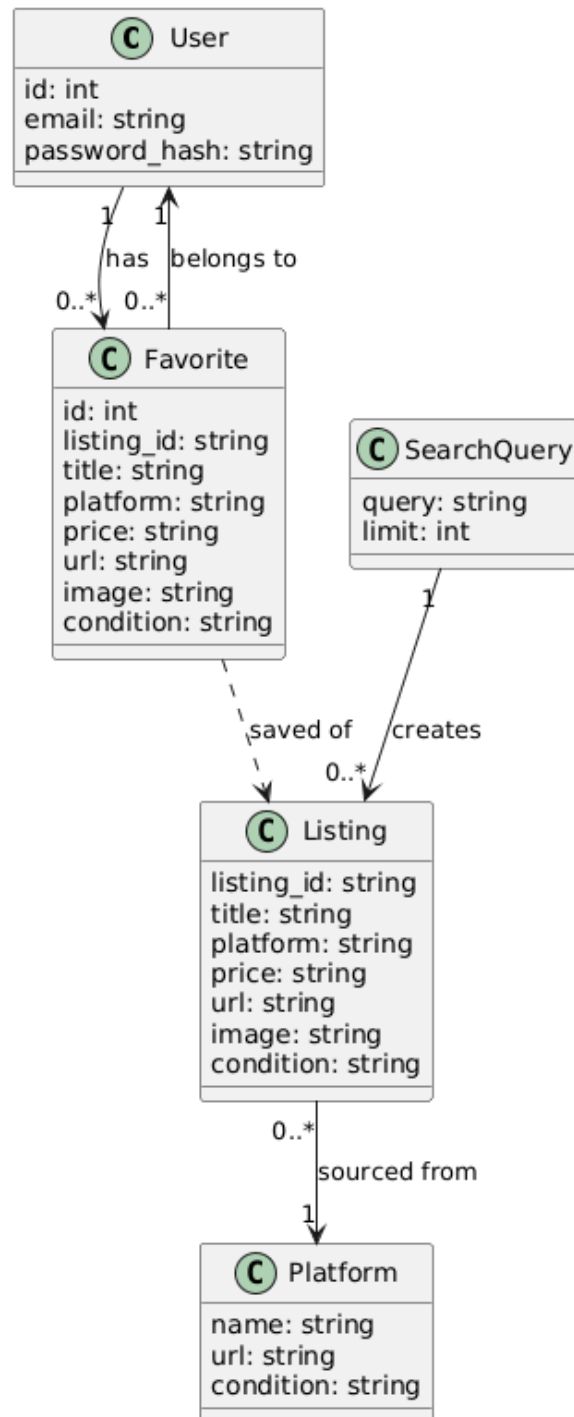


Figure 2: Domain Model Diagram

2.2 Conceptual Architecture

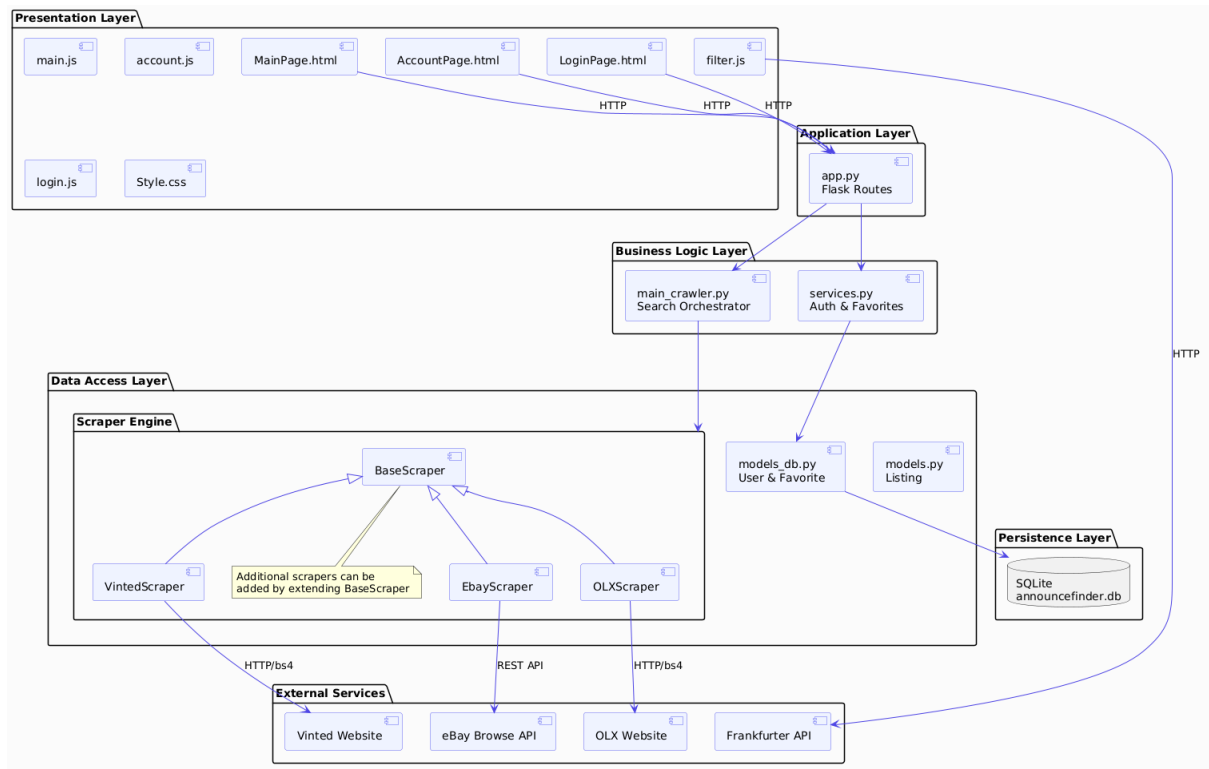


Figure 3: Conceptual Architecture Diagram

2.3 Package Design

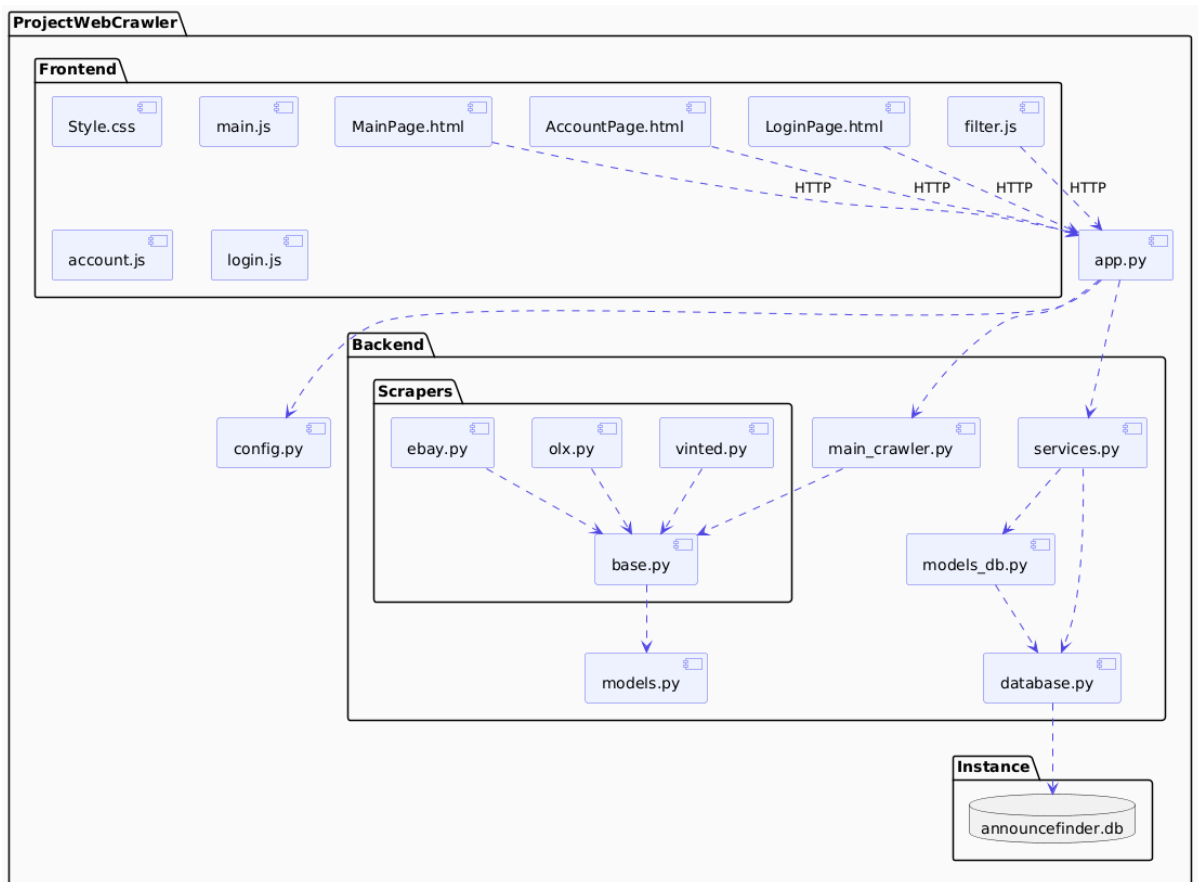


Figure 4: Package Diagram

2.4 Components and Deployment

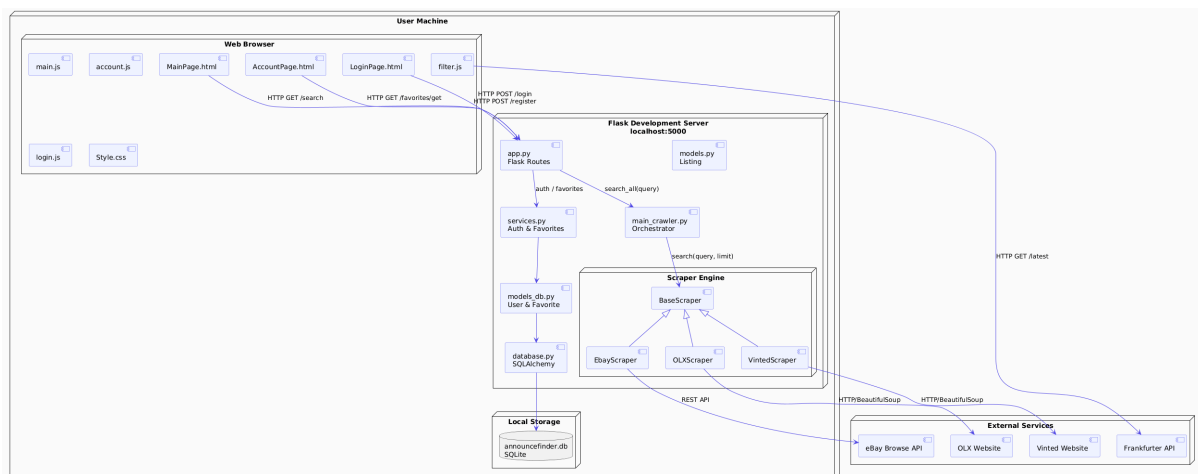


Figure 5: Component and Deployment Diagram

3 Deliverable 3

3.1 Design Model

3.1.1 Dynamic Behavior

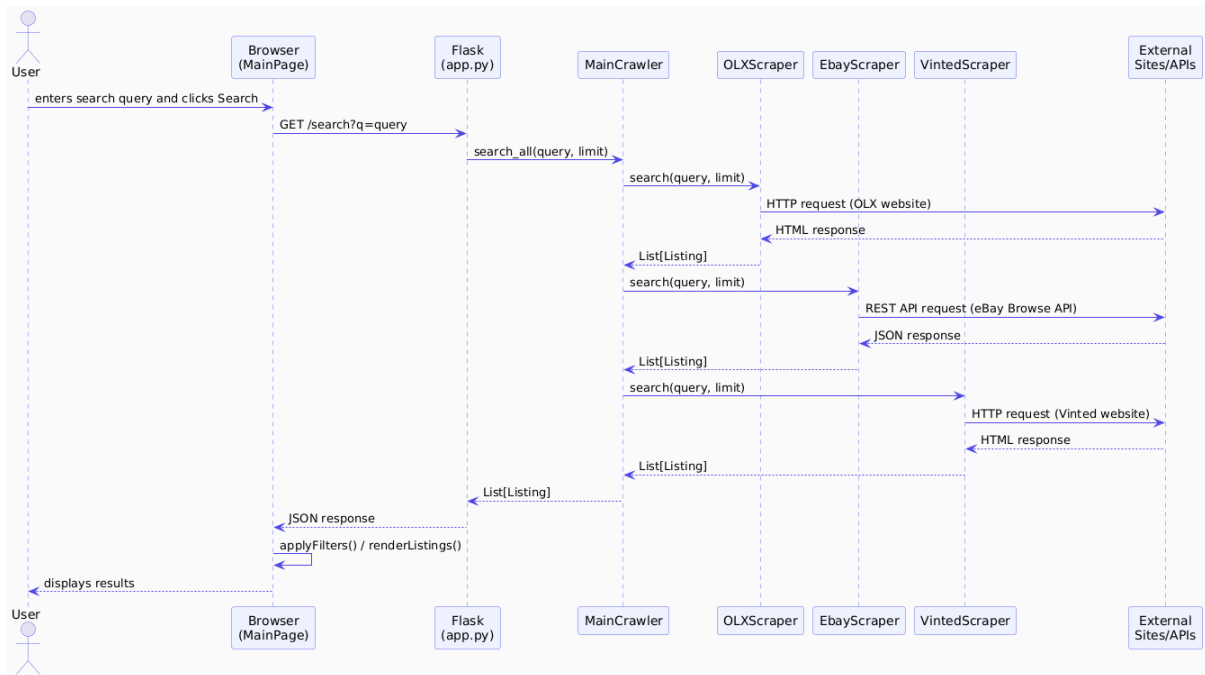


Figure 6: Sequence Diagram - Search for Listings

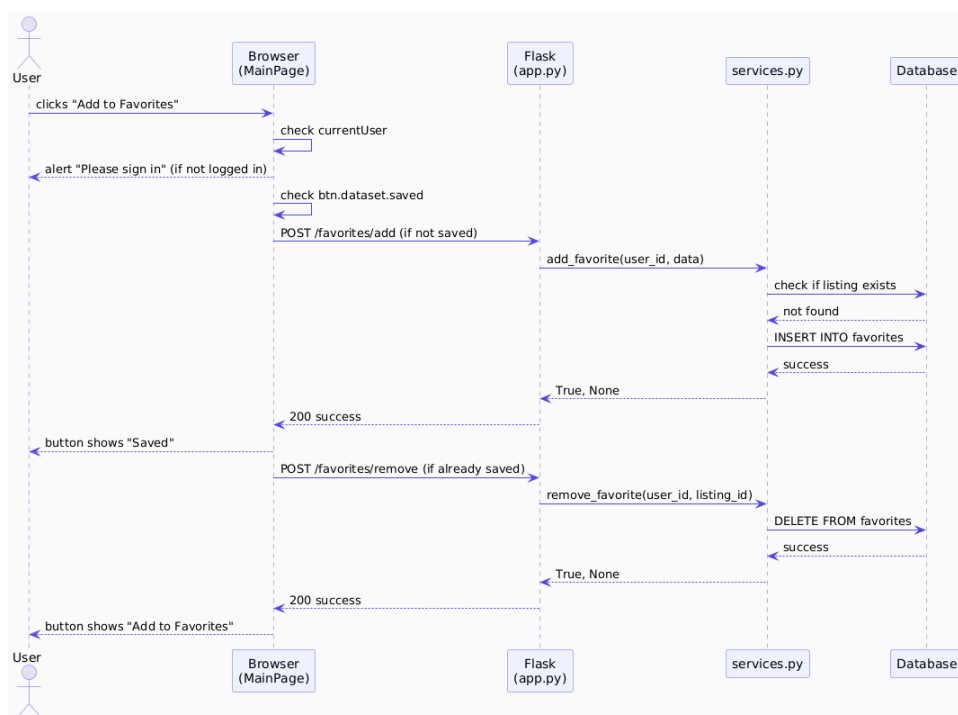


Figure 7: Sequence Diagram - Save a Listing to Favorites

3.1.2 Class Diagram

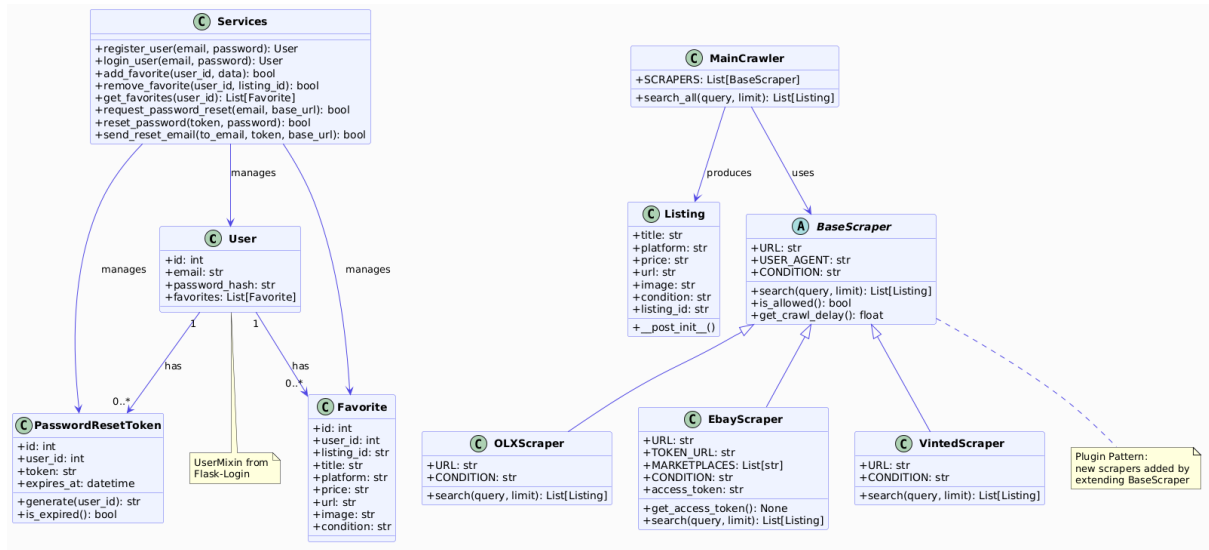


Figure 8: UML Class Diagram

3.1.3 Data Model

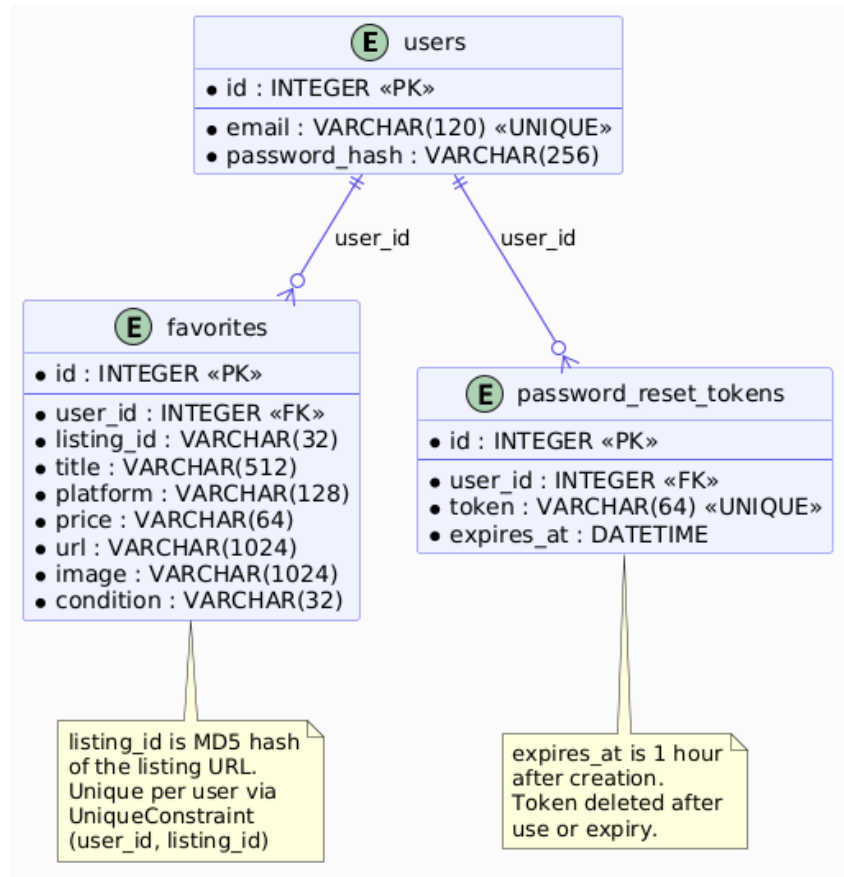


Figure 9: Data Model Diagram

3.1.4 System Testing

Testing Methods

The application was tested using manual black-box testing, where the system was evaluated from the user's perspective without knowledge of the internal implementation. Each feature was tested by interacting with the live application through the browser, verifying that inputs produced the expected outputs and that error cases were handled gracefully.

Test Cases

Test Case	Input	Output
Search for listings	Query: "warhammer"	Results displayed from OLX, eBay, Vinted
Search with no results	Query: "xyzxyzxyz"	"No results found" message
Register with valid data	Valid email and password	Account created, redirected to main page
Register with existing email	Already registered email	"Email already registered" error
Register with invalid email	"notanemail"	"Invalid email address" error
Login with correct credentials	Valid email and password	Logged in, redirected to main page
Login with wrong password	Valid email, wrong password	"Invalid email or password" error
Add listing to favorites	Click "Add to Favorites" while logged in	Listing saved, button shows "Saved"
Remove favorite	Click "Saved" button on saved listing	Listing removed, button resets
Filter by platform	Uncheck a platform checkbox	Listings from that platform hidden
Sort by price ascending	Select "Price: Low to High"	Listings sorted by price in EUR
Currency conversion	Select "RON" from currency dropdown	Prices displayed in RON
Password reset request	Enter registered email on forgot password page	Reset email sent
Password reset with valid token	Click reset link, enter new password	Password updated, redirected to login

Table 1: System Test Cases

3.1.5 Future Improvements

- **Email service migration** - The current password reset email functionality uses Gmail SMTP, which is blocked on Render's free tier. Migrating to SendGrid's API would resolve this limitation since it uses HTTPS instead of SMTP ports, works on all hosting tiers, and provides better deliverability and email tracking.
- **Additional scrapers** - The plugin architecture makes it straightforward to add

new platforms. Potential targets include AbeBooks for out-of-print books, Facebook Marketplace, and Romanian platforms such as eMag or Altex.

- **Price tracking** - Instead of only showing current listings, the system could periodically re-check saved favorites and notify users when the price of a saved listing drops.
- **Email notifications** - Users could opt in to receive email alerts when new listings matching their saved searches appear.
- **User preferences** - Allow users to set a default display currency, preferred platforms, and condition filters that persist across sessions instead of resetting on every page load.
- **Result caching** - Frequently searched queries could be cached for a short period to reduce the number of requests to external sites and improve response time.
- **Mobile application** - The current system is a web application. A dedicated mobile app would improve the user experience on smartphones, particularly for browsing and saving listings on the go.

3.2 Conclusion

AnnounceFinder is an online marketplace announcement aggregation service that collects and aggregates listings from several online marketplaces into one common interface. This project's design philosophy was scalability and maintainability, which led to developing an open-ended scraping system with the ability to extend functionality to include more online marketplaces with little effort.

All of the main functionalities outlined in the functional requirements were implemented in the program: multi-marketplace searching of OLX, Vinted, and eBay, authentication of users' accounts with password hashing, a favorites list managed with a PostgreSQL database, client-side sorting and filtering of products with live currency conversion, and a password reset method.

The application development cycle included implementing all layers, from backend scraping engines with REST APIs in Python and Flask, frontend interface in vanilla JavaScript, HTML, and CSS, to deploying a fully functional website to the live environment hosted with Render, using PostgreSQL as the production database.

Some technical issues were encountered while developing this project and had to be addressed. They include dealing with dynamic HTML structure in scraped websites, handling URL normalization of listings across multiple platforms, adjusting to network limitations within the free tier of Render, and making sure consistent functionality between the local Windows environment and the Linux production environment.

The project demonstrates a practical application of software engineering principles including layered architecture, the plugin design pattern, separation of concerns, and RESTful API design. It serves as a solid foundation for the future improvements outlined above.

3.3 Bibliography

References

- [1] Cloudflare. *What is a web crawler?* Available at: <https://www.cloudflare.com/learning/bots/what-is-a-web-crawler/> [Accessed: March 2026]
- [2] Hello Interview. *Web Crawler System Design*. Available at: <https://www.hellointerview.com/learn/system-design/problem-breakdowns/web-crawler> [Accessed: March 2026]
- [3] Pallets Projects. *Flask Documentation*. Available at: <https://flask.palletsprojects.com/> [Accessed: March 2026]
- [4] SQLAlchemy. *SQLAlchemy Documentation*. Available at: <https://docs.sqlalchemy.org/> [Accessed: April 2026]
- [5] eBay Inc. *eBay Browse API Documentation*. Available at: <https://developer.ebay.com/api-docs/buy/browse/overview.html> [Accessed: March 2026]
- [6] Frankfurter. *Frankfurter Exchange Rates API*. Available at: <https://www.frankfurter.app/> [Accessed: April 2026]
- [7] Leonard Richardson. *Beautiful Soup Documentation*. Available at: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> [Accessed: March 2026]
- [8] Render. *Render Documentation*. Available at: <https://render.com/docs> [Accessed: May 2026]